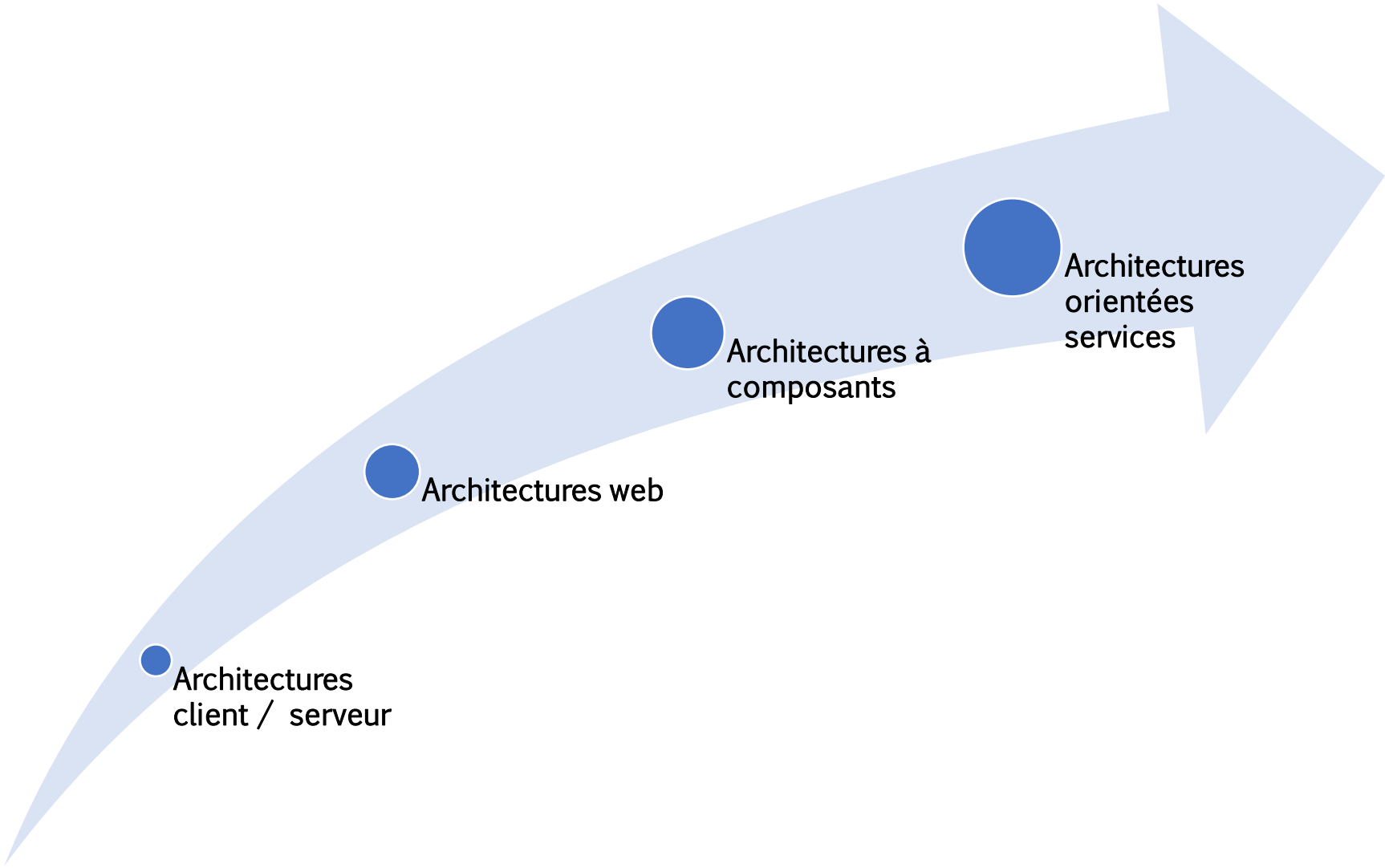
Chapitre 1: Introduction aux Architectures Orientées Services

hamdi.aloulou@isima.u-monastir.tn

Vers les architectures orientées services



Vers les Architectures Orientées Services Architecture Client-Serveur

› Définition

Approche d'architecture qui vise à répartir un système informatisé sur des machines distantes, en utilisant des protocoles standards et grâce à des matériels banalisés.

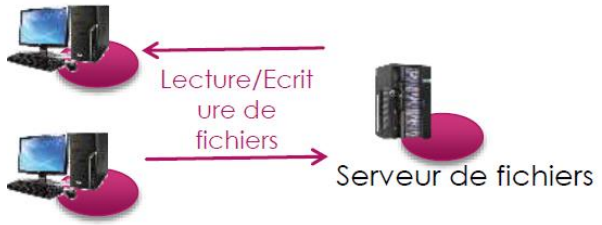
› Deux types de logiciels :

- Logiciels clients
- Logiciels serveurs

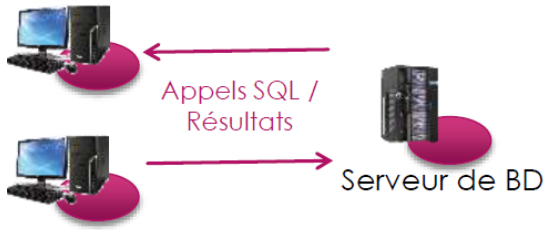
Vers les Architectures Orientées Services
Architecture Client-Serveur

› Modèles Client-Serveur

- Modèle serveur de fichiers
  Partage de fichiers sur un réseau



- Modèle serveur de bases de données relationnelles
  Requêtes distantes sur une BD



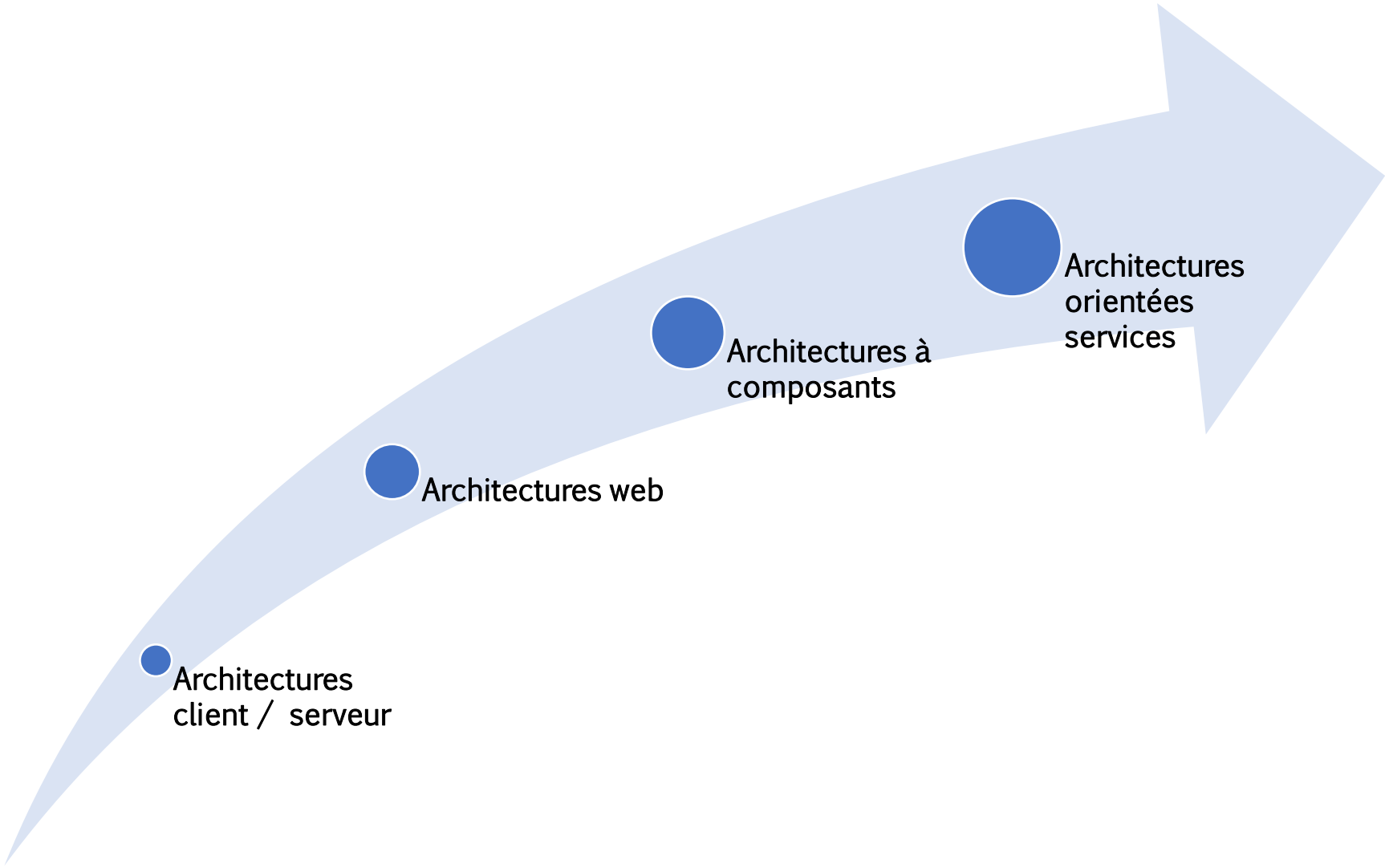
- Modèle serveur de transactions
  Applications à temps de réponse critique



Vers les Architectures Orientées Services Architecture Client-Serveur

- **Caractéristiques**
  - Partage des ressources / Distribution de la charge
  - Intégrité des données partagées
  - Transparence relative à la localisation
  - Capacité d'évolution du système
    - Ajout de stations clientes
    - Changement de serveur

Vers les architectures orientées services



Vers les Architectures Orientées Services
Les Architectures Web

- Interconnexion de réseaux à **l'échelle mondiale fondée sur les protocoles TCP/IP**
- Exploitation des **technologies internet** dans le modèle client-serveur
- Évolution du client/serveur en local vers **le client/serveur sur Internet**
- Modèle du « **client léger** » et **Architecture 3 tiers**

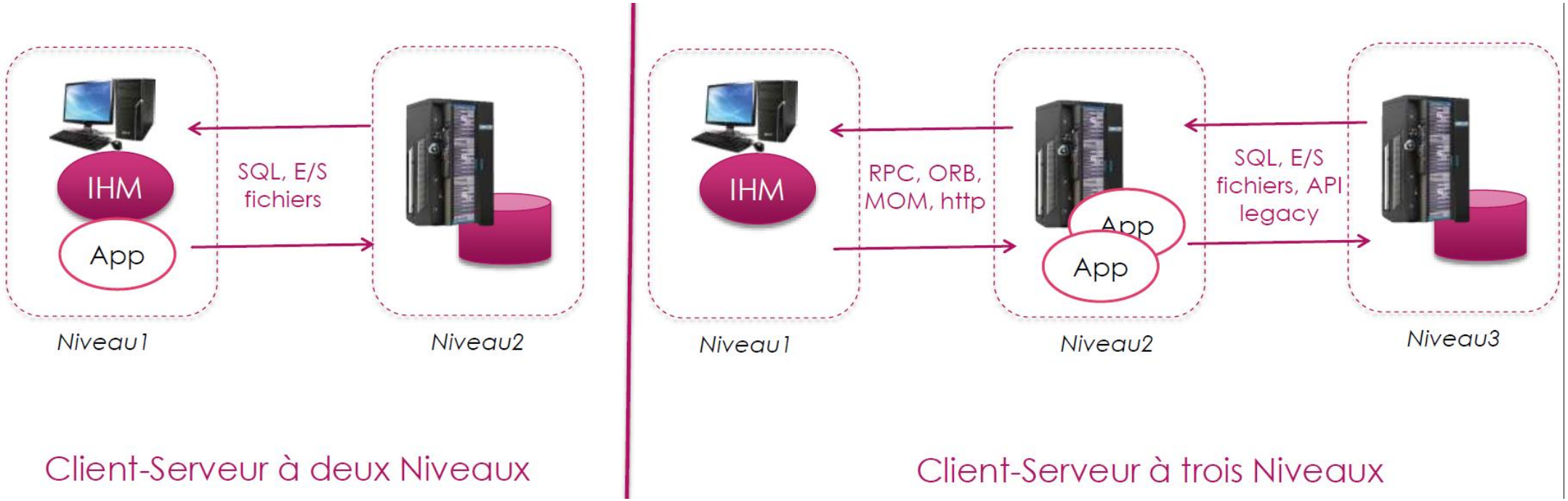
Vers les Architectures Orientées Services
Les Architectures Web

Architecture 3 tiers

- Principe : séparation des trois niveaux
  - IHM : interface Homme-Machine
  - Application
  - Gestion des données
- Les applications peuvent être déployées et administrées de manière indépendante des IHM
- Placement des serveurs logiciels sur un ou plusieurs serveurs physiques

Vers les Architectures Orientées Services
Les Architectures Web

Passage du 2 tiers au 3 tiers



Vers les Architectures Orientées Services
Les Architectures Web : Réseau d'ordinateurs

- Un serveur est un ordinateur connecté en permanence à l'Internet et exposé avec une adresse IP publique
  - De la forme : 178.12.77.1
- Il met à disposition des informations ou des applications
- Serveur de DNS (ou annuaire)
  - Transforme www.site.com en 178.12.77.1

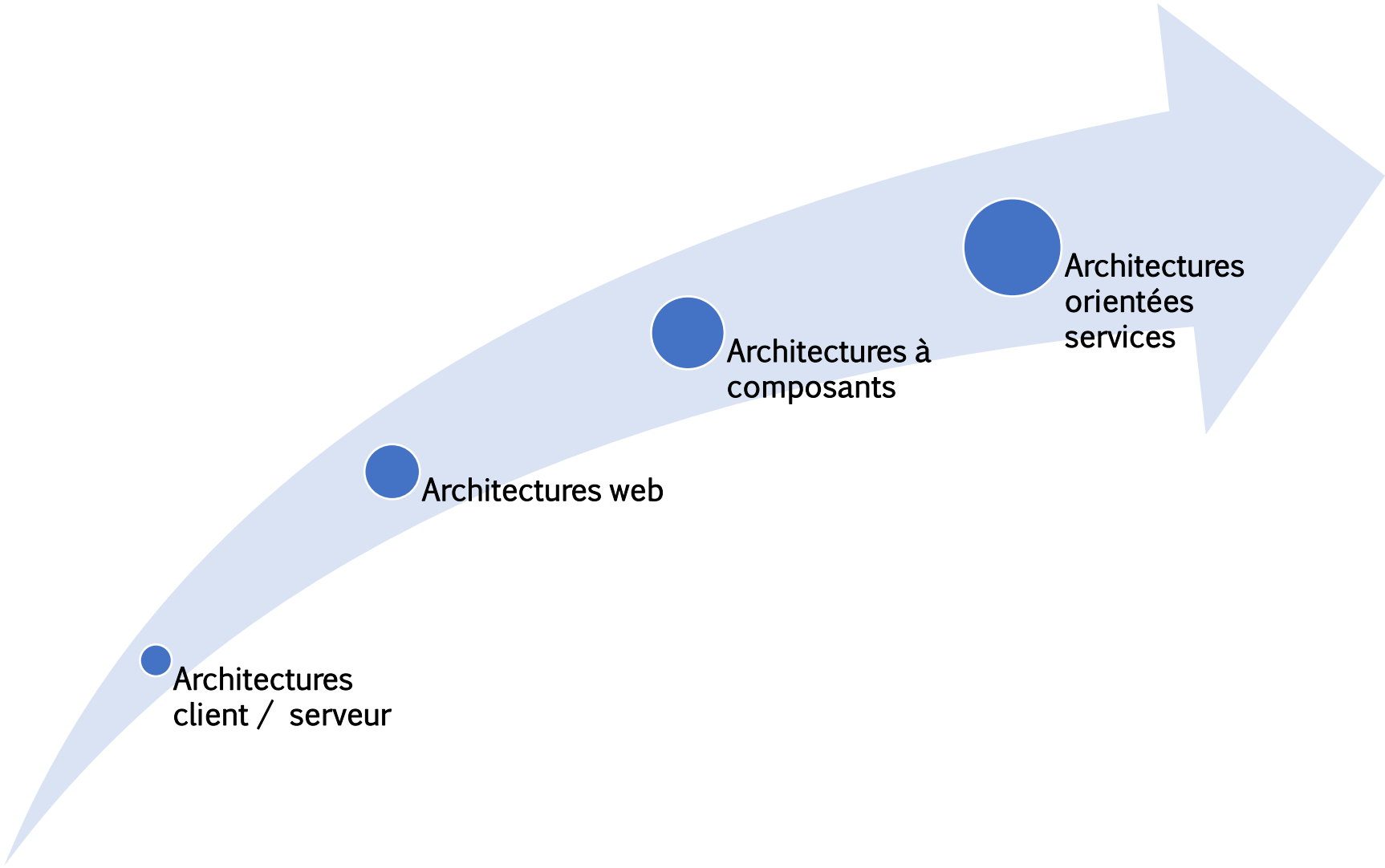
Vers les Architectures Orientées Services
Les Architectures Web : Besoin de protocole

- Nécessité de parler un langage commun pour pouvoir communiquer dans un réseau
- Un protocole présente des formats de dialogue, des règles d'échange et de présentation de données, etc.
  - Les protocoles HTTP, HTTPS, FTP, IMAP, SMTP, etc.

Vers les Architectures Orientées Services
Les Architectures Web : Besoin de protocole

- **HTTP :** Protocole utilisé pour transférer des documents hypertextes ou hypermédias entre un serveur et un client Web.
- **IMAP :** Protocole de messagerie électronique qui garantit aux utilisateurs la livraison des messages, filtre les courriels et délivre automatiquement des accusés de réception.
- **SMTP :** Protocole utilisé pour l'envoi de messages entre serveurs. Il s'agit du standard de facto pour l'envoie de message d'un client vers un serveur ou d'un serveur à l'autre.
- **FTP :** Protocole de transfert de fichiers qui permet de télécharger une copie de données choisies par l'internaute, d'un ordinateur à un autre, selon le modèle client-serveur.

Vers les architectures orientées services



Vers les Architectures Orientées Services
Les Architectures à Composants

Principe :

› Construction d'applications à partir de l'**assemblage de composants**

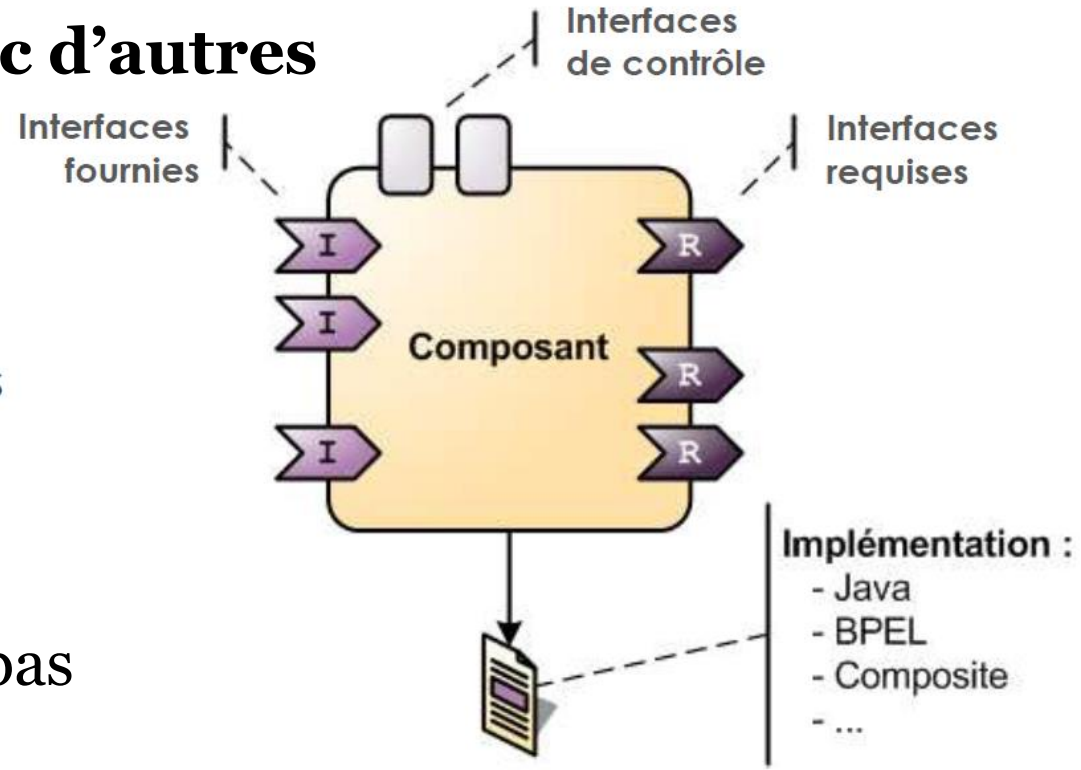
› Le développement et assemblage de composants peut être réalisé séparément par des acteurs différents à des endroits différents

› Utilisation de la **composition** comme mécanisme de réutilisation

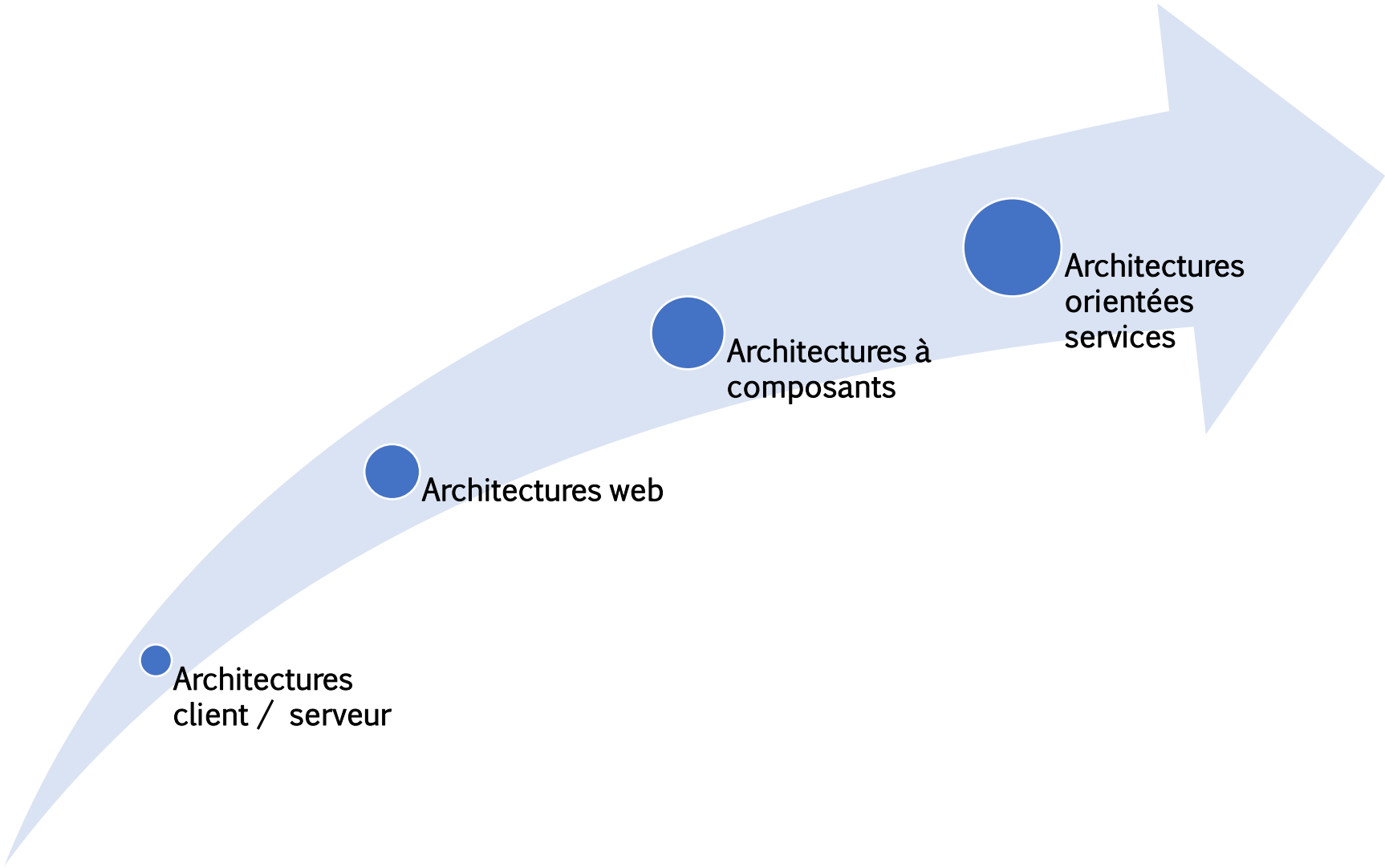
Vers les Architectures Orientées Services
Les Architectures à Composants

Les Composants :

- Briques logicielles préfabriquées conçue pour être **composée (assemblée) avec d'autres composants**
- **Réutilisables**
- Leur utilisation ne nécessite pas de connaissance sur l'implémentation
- Unités binaires : code source n'est pas forcément livré avec le composant



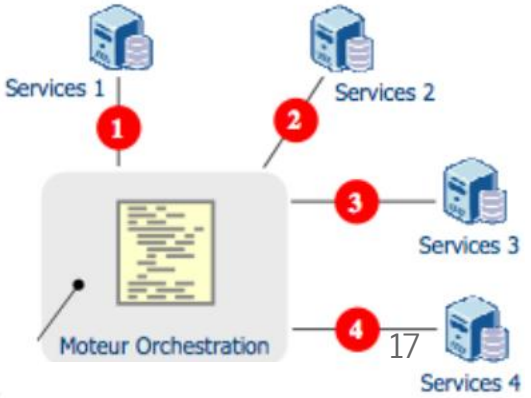
Vers les architectures orientées services



Définition de Architectures Orientées Services

› SOA (Service Oriented Architecture)

- Permet de **décomposer** une fonctionnalité en un ensemble de **fonctions basiques** (**services web**) **fournies par des composants**
- Permet d'**intégrer et de manipuler** les différentes briques et composants applicatifs d'un système informatique et de **gérer les liens** qu'ils entretiennent.
- **Décrit** finement le **schéma d'interaction entre ces services**



Principes de la SOA

- **"Diviser pour régner"**

  Substituer la découpe strictement applicative par une structuration en composants plus réduits et potentiellement plus simples à faire évoluer.

- **Alignement métier**

  Construire et organiser le système à partir des réalités métiers, qui doivent se retrouver dans ses constituants.

Principes de la SOA

- **Neutralité technologique**

  Assurer une indépendance totale entre les interfaces et les implémentations.

  L'élément qui utilise un service ne doit pas être contraint ni par la technologie d'implémentation, ni par sa localisation (potentiellement distribué).

Principes de la SOA

- **Réutilisation**

  Favoriser la réutilisation de services métiers par plusieurs lignes métiers ou applications.

  Permettre la construction de services de haut niveau par combinaison de services existants.

Les Architectures Orientées Services : Tout devient « service »

- **Service**
  - **Périmètre fonctionnel** qu'on souhaite **exposer à** un certain type de **consommateurs**
  - Fonctionnalité **réutilisable** dont le comportement est défini de façon contractuelle
  - Expose un petit nombre d'opérations **offrant un traitement de bout en bout.**

Définition d'un Web Service

- **Composant logiciel** fourni sur le web limité à une **opération élémentaire** accessible en utilisant **les protocoles standards d'internet**
- Possède une **interface** pouvant être **découverte dynamiquement** et qui le décrit dans un **format pouvant être traité par une machine**.
- Exposé sur le net, **par et pour des applications ou machines**, sans intervention humaine, de manière synchrone ou asynchrone.

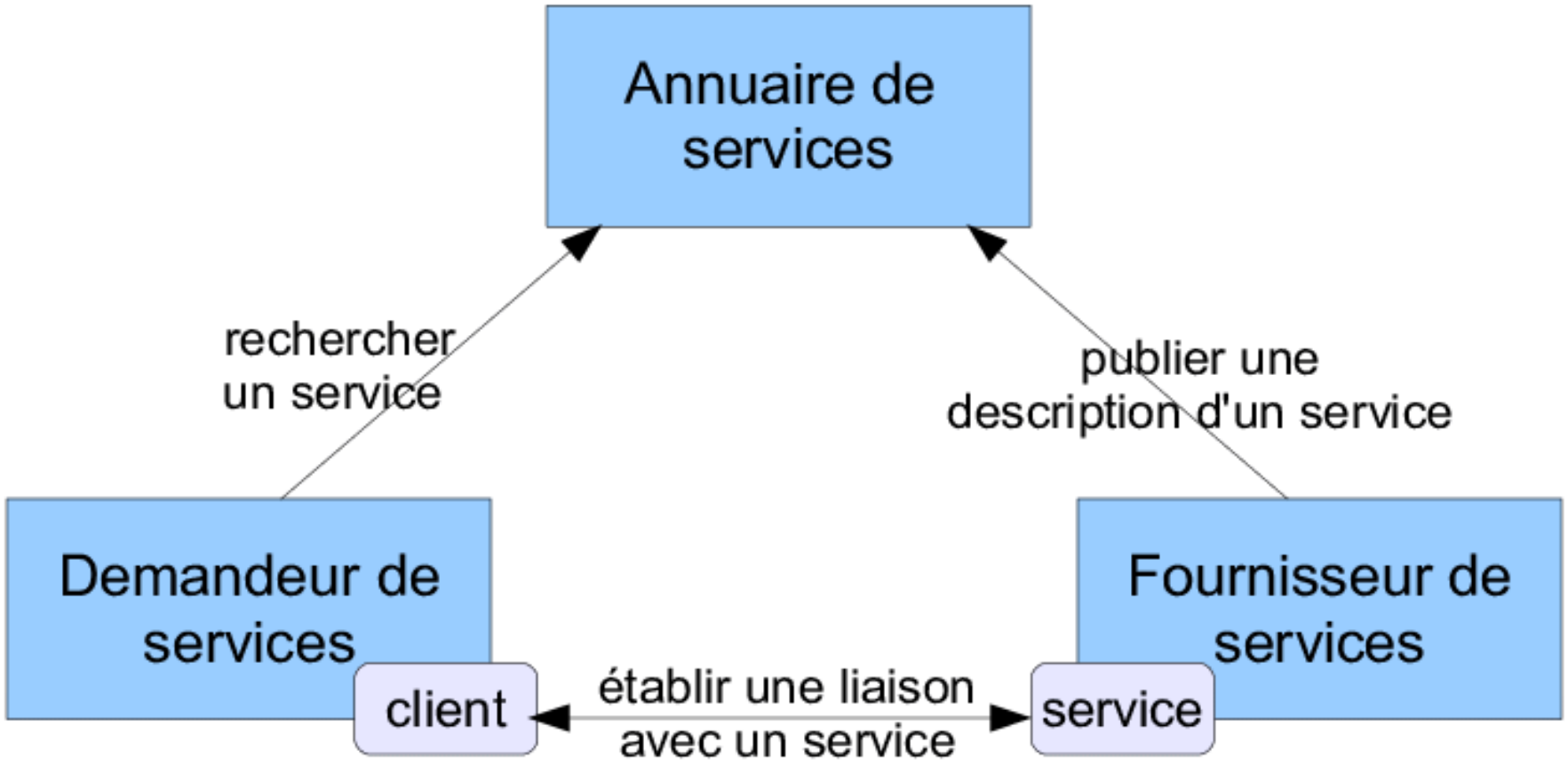
Définition d'un Web Service

- **Peut être invoqué** par d'autres services ou applications clientes
- Permet la **communication et l'échange de données** entre applications et systèmes hétérogènes (**machine-to-machine**) dans des **environnements distribués indépendamment des plates-formes** et **des langages** sur lesquelles elles reposent.
- Les requêtes et les réponses s'effectuent dans des **formats ouverts (HTML, XML, JSON ou text)** et transitent par Internet.

Les Architectures Orientées Services : Trois acteurs

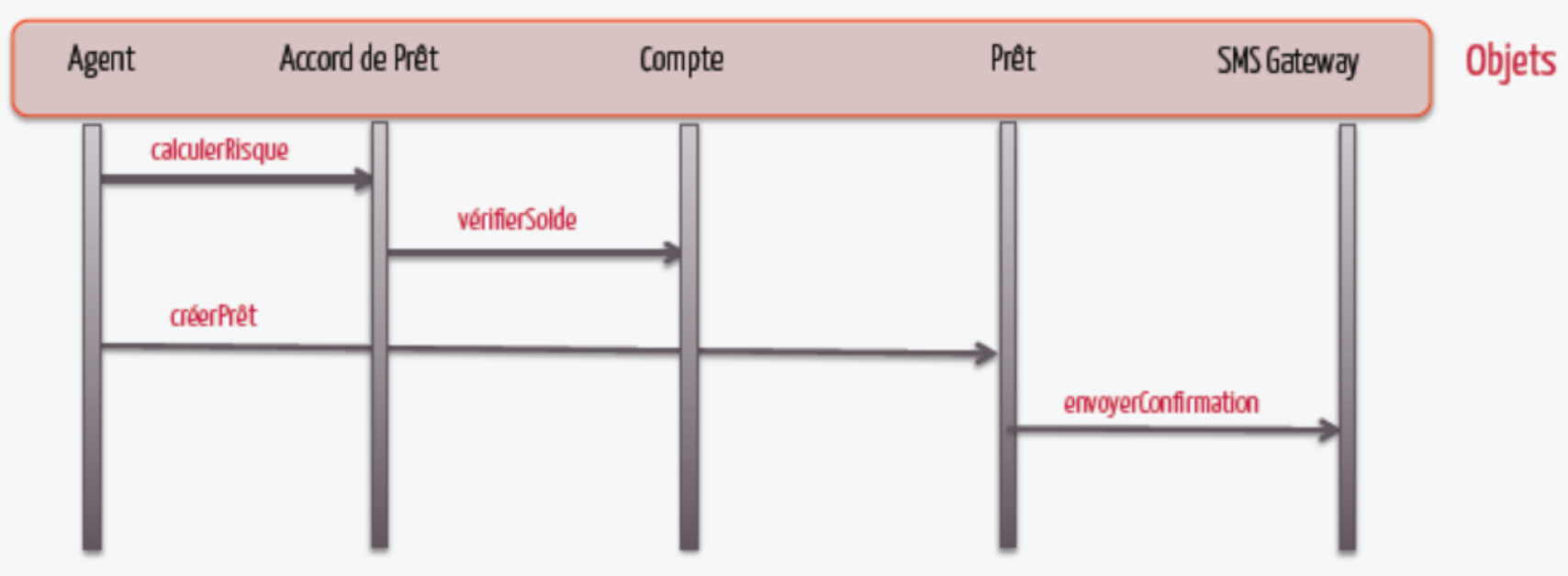
- **Consommateur de services** décrit le service à consommer
- **Fournisseur de services** découvert en temps d'exécution à l'aide d'un intermédiaire (*Registre de service*)
- **Annuaire de services** contient l'ensemble des descripteurs de services et les références vers les fournisseurs de services

Les Architectures Orientées Services : Trois acteurs



Les Architectures Orientées Services Couplage Fort Vs. Couplage Faible

› **Couplage Fort**

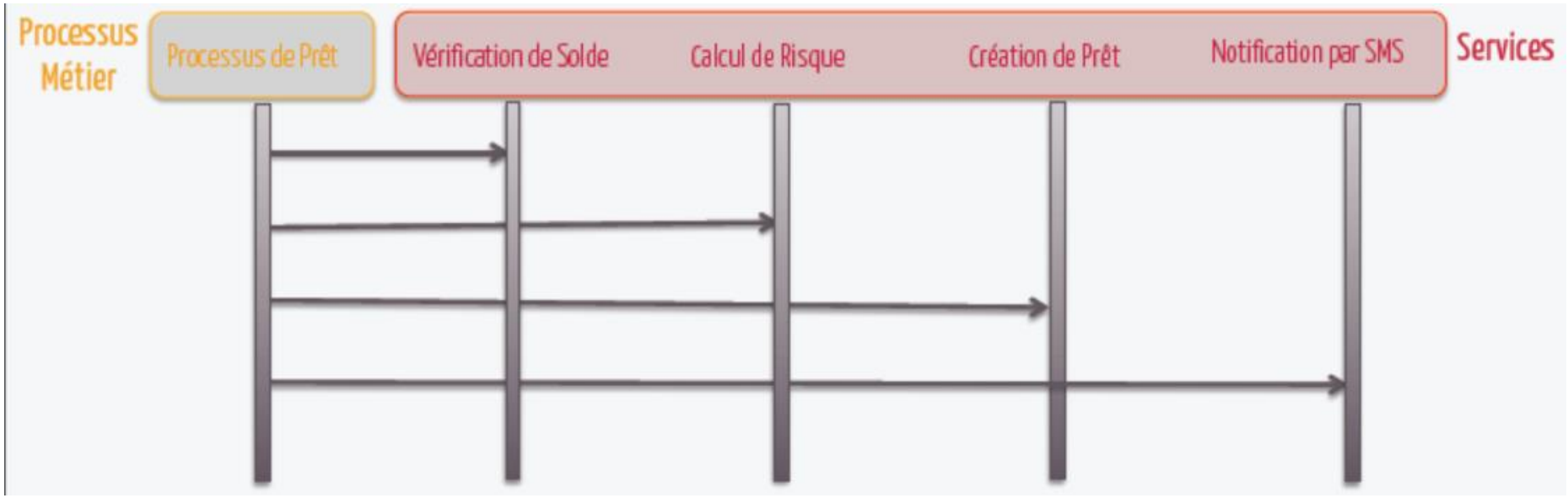


› «Agent » est lié à « Accord de Prêt », qui est lié à « Compte »

› « Agent » est lié à « Prêt » qui est lié à « SMS Gateway »

Les Architectures Orientées Services Couplage Fort Vs. Couplage Faible

- **Couplage Faible**



- Chaque entité (service) a un fonctionnement indépendant des autres
- Le processus métier « Processus de Prêt » permet d'orchestrer les services

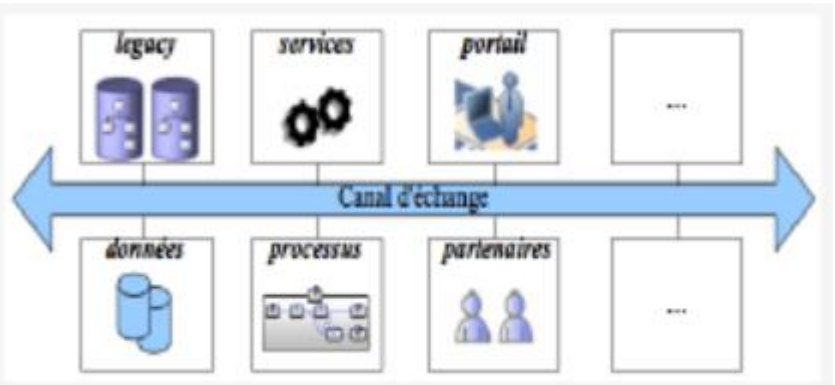
Les Architectures Orientées Services Couplage Fort Vs. Couplage Faible

Hier : Architecture en spaghetti



- Développement coûteux
- Interconnexions redondantes
- Grande complexité
- Réutilisation et maintenance difficile

Demain : Architecture urbanisée



- Découper le SI en modules autonomes
- Localiser les zones d'échange d'informations

Types des Services Web

- Il existe deux types de Services Web les plus connus :
  - Les Services Web REST (aussi appelés Web API)
  - Les Services Web SOAP (aussi appelés Big Web Services)

Apport des Services Web

- S'appuient sur **des standards répandus**
  - Format de données : **XML, XML Schéma**
  - Transport : **HTTP, HTTPS, SMTP, FTP**

    SOAP, XML-RPC
- Description de service : **WSDL, WSCL** : définit les interfaces abstraites du service web

Apport des Services Web

- Découverte, Publication : **UDDI** (Universal Description Discovery and Integration) ⇨ Meilleure interopérabilité
- Composition : **BPEL**
- Ne sont pas centrés sur un langage ou technologie
  - On peut faire communiquer du .Net avec du Java ou du Python

Le protocole HTTP

Protocole HTTP

- HTTP => HyperText Transfer Protocol
- Protocole d'échanges d'information sur le web
- Protocole qui permet au client de récupérer des documents du serveur
- Ces documents peuvent être statiques (contenu qui ne change pas : HTML, PDF, Image, etc..) ou dynamiques (Contenu généré dynamiquement au moment de la requête : PHP, JSP, ASP…)

Le Protocole HTTP

- **Fonctionnement (très simple; Protocole sans état)**
  - Le client se connecte au serveur
  - Le client demande au serveur un document : **Requête HTTP**
  - Le serveur renvoi au client le document (status=200) ou une erreur (status=404 quand le document n'existe pas) : **Réponse HTTP**
  - Déconnexion

Enchainement Client - Serveur

(1) L'utilisateur lance une URL à travers le navigateur http://host:port/path/file

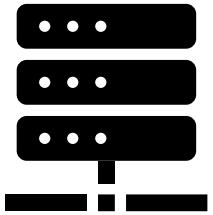


Client (Browser)

(2) Le navigateur envoi une requête HTTP

GET URL HTTP/1.1
Host: host:port
……
……

(3) Le serveur mappe l'URL vers un fichier ou le répertoire du document



Server (@host:port)

(4) Le serveur envoi une réponse HTTP

HTTP/1.1 200 OK
……
……
……

(5) Le navigateur formatte la réponse et l'affiche

HTTP (over TCP/IP)

URL (Uniform Resource Locator)

- Une URL (Uniform Resource Locator) est une chaîne de caractères utilisée pour identifier les ressources de manière unique sur le Web
- 4 parties
  - Protocole (http, ftp, mail, …)
  - Host (google.com)
  - Port (8080, 80)
  - Path (Chemin vers la ressource sur le serveur)

URL :

Protocole://Host:Port/Path

URL (Uniform Resource Locator)

› Exemple :
http://www.example.com:8080/chemin/page.html

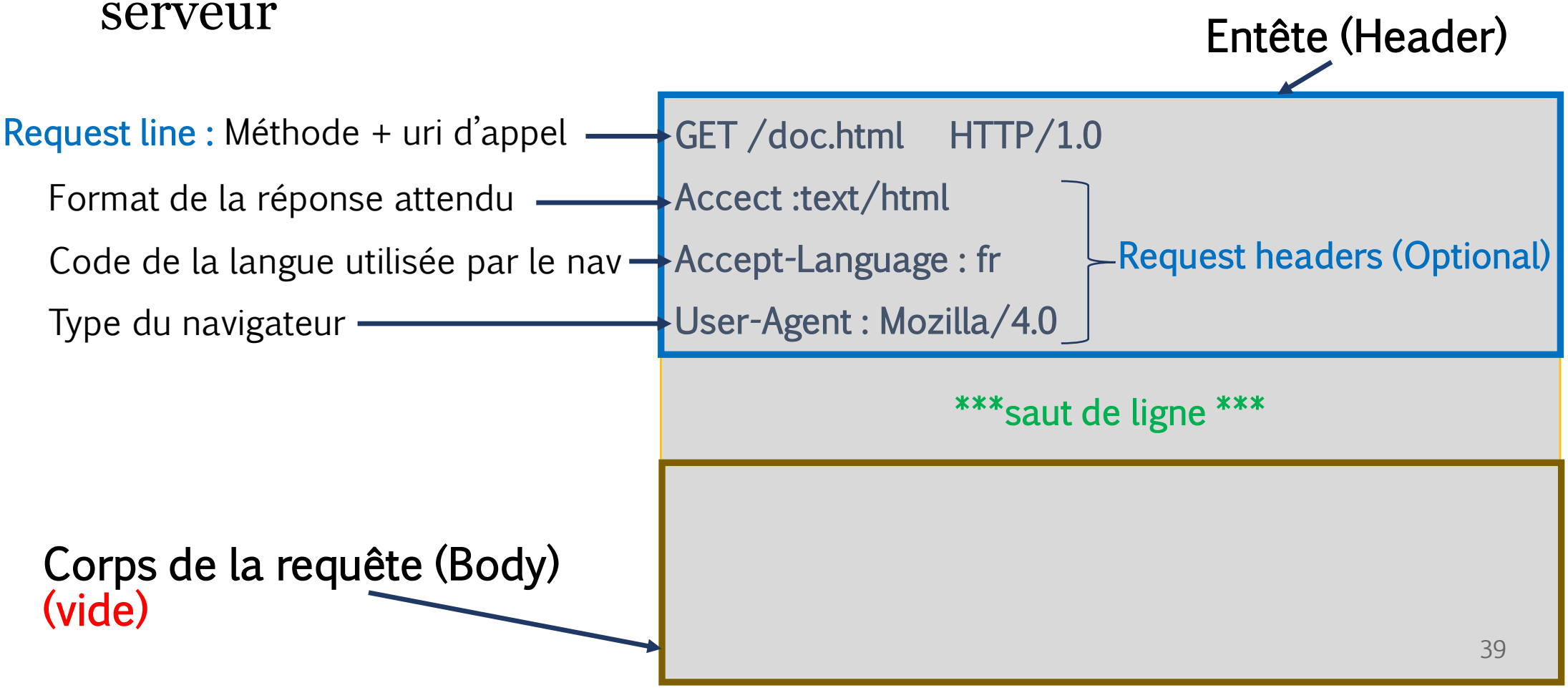
- http : protocole
- www.example.com : hôte
- 8080 : numéro du port
- /chemin/ : chemin absolu vers la page
- page.html : nom de la page Web demandée

Les méthodes du protocole HTTP

- Une requête HTTP peut être envoyée en utilisant les méthodes suivantes :
  - **GET :** Pour récupérer le contenu d'un document
  - **POST :** Pour soumissionner des formulaires (Envoyer, dans la requête, des données saisies par l'utilisateur)
  - **PUT :** Pour envoyer un fichier du client vers le serveur
  - **DELETE :** Pour demander au serveur de supprimer un document.
  - **HEAD :** Pour récupérer les informations sur un document (Type, Capacité, Date de dernière modification, etc…)

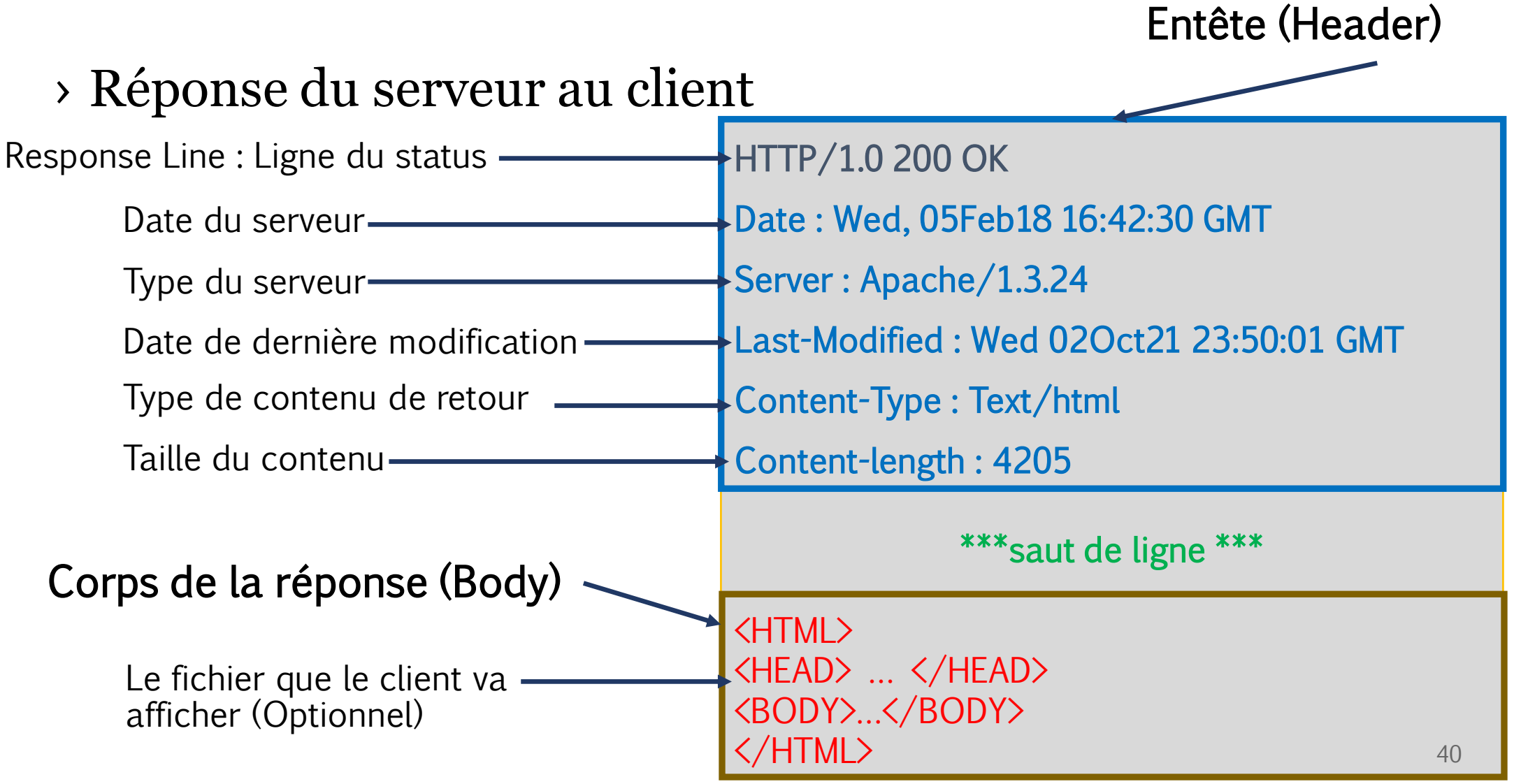
La requête HTTP

› Permet à un client de demander une ressource sur un serveur



La réponse HTTP

› Réponse du serveur au client



Le Protocole HTTP : Les codes de réponse

- Les codes de réponse se divisent en 5 classes :
  - **1XX :** Informationnel
    - Indique une réponse provisionnelle
    - Exemples : 100 : Continue
  - **2XX** : Succès
    - Indique que la requête a été reçu, compris, accepté et traité avec succès
    - Exemples : 200 : OK, 201 : Created.
  - **3XX** : Redirection
    - Indique que d'autres mesures doivent être prise pour répondre à la requête.
    - Exemple : 301 : Moved Permanently
  - **4XX :** Erreur Client
    - Utiliser dans les cas où le client a fait une erreur
    - Exemples : 403 : Forbidden, 404 : Not found, 405 : Method Not Alloweds
  - **5XX** : Erreur Serveur
    - Indique que le serveur est conscient qu'il a rencontré une erreur ou est incapable d'effectuer la demande.
    - Exemples : 500 : internal Server Error, 501 : Not Implemented